DAY 3 | HANDS-ON ACTIVITIES

OOP and OpenAI Codex Walkthrough: Problem Statements

Eight problems. Problems 1 to 4 run in the Day 3 notebook. Problems 5 to 8 use OpenAI Codex in VS Code; write a short review note for each.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

How to work: read the problem, write the steps in plain English first, then write the code. Test with the sample run, then try your own inputs, including wrong ones.

Problem 1: Student Class

Level: Warm-up | **Concepts:** class, __init__, methods, __str__

Create a Student class with name and a list of marks. Add add_mark(m) and average(). Add __str__ so print(student) shows name and average.

Sample run / expected result

```
s = Student("Aiko")
s.add_mark(80); s.add_mark(90)
print(s)   # Aiko (avg 85.0)
```

Hints

- Start marks as an empty list inside __init__.

Problem 2: Customers and Polymorphism

Level: Easy | **Concepts:** inheritance, overriding, super()

Create a Customer class with monthly_fee() returning 500. Create PremiumCustomer (half price, using super()) and StaffCustomer (free). Loop over a list of customers and print each fee.

Sample run / expected result

```
Customer         500
PremiumCustomer  250
StaffCustomer    0
```

Hints

- type(obj).__name__ gives the class name.

Problem 3: Bank Account with Rules

Level: Medium | **Concepts:** encapsulation, custom exceptions, finally

Build BankAccount with deposit and withdraw. Raise InsufficientFunds when withdrawing too much and ValueError for negative amounts. Keep a transaction history list.

Sample run / expected result

```
acc.withdraw(5000)
InsufficientFunds: Balance 1200, requested 5000
```

Hints

- class InsufficientFunds(Exception): pass

Problem 4: Pin and Check Your Dependencies NEW (v4)

Level: Easy | **Concepts:** venv, requirements.txt, pip-audit

Given a requirements list, write check_pinned(lines) that returns the packages that are not pinned with ==. Then, in VS Code, create a venv for your SmartDesk folder, pin your packages and run pip-audit.

Sample run / expected result

```
check_pinned(["requests==2.32.3", "pandas", "numpy>=1.26"])
-> ['pandas', 'numpy>=1.26']

$ pip-audit -r requirements.txt
No known vulnerabilities found
```

Hints

- A line is pinned only if it contains ==.
- Add .venv/ to .gitignore.

Problem 5: Codex on Your Own Day 2 Code NEW (v4)

Level: Medium | **Concepts:** Codex explain, generate, review

Open your Day 2 expense tracker or ticket logger in VS Code. Ask Codex to (1) explain one function, (2) add input validation to one function with a test. Read the diff before accepting.

- Write a review note: what Codex changed, what you accepted, what you rejected and why.
- Record the tokens used from your usage page or the Codex panel.

Sample run / expected result

```
Review note: Codex added a date check and one test. Accepted the check. Rejected a new
dependency (python-dateutil) because datetime is enough. Tokens used: about 3,400.
```

Hints

- Ask Codex for a plan first.
- Include only the file you are changing.

Problem 6: SmartDesk: Search Tickets with Codex NEW (v4)

Level: Medium | **Concepts:** structured AI-assisted coding

SmartDesk step 2. Convert tickets to Ticket, Customer and Agent classes. Then use Codex to add search_tickets(tickets, text): case insensitive, matches customer name or issue, returns a list.

- Describe the task, ask for a plan, approve it, then let Codex build it in small steps.

- Ask Codex to write three tests first. Run them. Review the diff.
- Write a one-line review note and the tokens used.

Sample run / expected result

```
search_tickets(tickets, "SIGNAL") -> [#1 Mei: No signal at home]
search_tickets(tickets, "zzz")    -> []
3 passed
```

Hints

- Use text.lower() in both places.
- Reject any use of eval() in the generated code.

Problem 7: Spot the Security Problems NEW (v4)

Level: Challenging | **Concepts:** security of AI-generated code

The notebook contains three functions "written by an AI tool". Find every security problem (for example eval, shell=True, a hard-coded key, SQL built with f-strings), explain the risk and rewrite each function safely.

Sample run / expected result

```
1. eval(user_text)        -> code injection. Use int() or a lookup table.
2. API_KEY = "..."         -> secret in code. Read from the environment.
3. f"SELECT ... {name}"    -> SQL injection. Use parameters.
```

Hints

- Run the notebook's simple scanner first, then read the code yourself.

Problem 8: Codex Cost Estimate NEW (v4)

Level: Stretch | **Concepts:** tokens, model choice, daily budget

Use the notebook's token estimator to estimate tokens for three Codex tasks of different sizes. Pick a model tier and reasoning effort for each, and estimate the daily cost with the placeholder prices.

Sample run / expected result

```
explain one function  : light,   low    ->  2,100 tokens
add search + tests    : default, medium ->  9,800 tokens
refactor 6 files      : default, high   -> 41,000 tokens
```

Hints

- Prices are placeholders. Label them.
- Fewer files in context means fewer tokens.